# SOFTWARE QUALITY AND TESTING

## PRACTICAL PROJECT TEST 1

## TicketHub QA Package

Prepared by

**OTIM MICHEAL**

SEP24/BSE/4497U

Repository : https://github.com/micheal-61/tickethub-qa

Requirements-based test design • Pytest automation • Defect analysis • Release assessment

# Part A: Foundations of Software Quality

## A1. Views of Quality

| View | Quality criterion | How measured / observed | QA or QC activity |
|---|---|---|---|
| Product | Every calculate_total result matches appendix A to the cent for all discount rules | Automated pytest assertions on decimal equality across the decision-table rows | **QC:** execute the regression suite in CI on every push |
| Process | 100% of REQ-F1..F7 map to ≥1 test case; no unmapped functional requirement | Review the traceability matrix at the end of Day 6 | **QA:** peer review of the test plan and matrix before execution |
| User | A buyer can pay, collect, or cancel and always sees the correct final state | Exploratory session + acceptance walk-through of pay→collect and pay→cancel→refund | **QC:** acceptance test with the event organiser |
| Value based | The discount charged matches what the organiser agreed to sell (never over- or under-discounts a student/staff/public ticket) | Stakeholder review of the collapsed decision table against the signed discount policy | **QA:** requirements review with the organiser before any test is written |

## A2 Classifying the activities

| Activity | V / V | QA / QC | Testing / Debugging / Neither |
|---|---|---|---|
| Review requirements with organiser | validation | QA | Neither (requirements engineering) |
| Run suite in CI on every push | Verification | QC | Testing |
| No merge without peer review | Verification | QA | Neither (process control) |
| Step through calculate_discount in IDE | Verification | QC | **Debugging** |
| Inspect requirements for ambiguity | Verification | QA | Neither (static review) |
| Pilot with ten students, collect feedback | validation | QC | Testing (acceptance) |
| Audit adherence to documented test procedure | Verification | QA | Neither (process audit) |
| Change code after finding root cause | Verification | QC | **Debugging** |

**Justifications (rows 1, 4, 7):**

**Row 1 — Validation / QA:** Confirming the requirements describe what the organiser actually needs is validation (building the right product). Preventive QA: it stops defects entering the codebase. Not testing — no artifact is executed.

**Row 4 — Verification / Debugging:** Stepping through code to find why a total is wrong is verification and is debugging. Not testing — no test case with a defined expected result is being run.
**Row 7 — QA / Neither:** Auditing adherence to a documented test procedure is process assurance. Neither testing nor debugging — no product artifact is executed.

## A3. Testing in Process Models

| Aspect | V model | Agile / DevOps |
|---|---|---|
| Test design starts | In parallel with each development level (unit during detailed design, acceptance during requirements) | Continuously; alongside each user story, often test-first (TDD) |
| Execution happens | After the matching build level completes, in strict sequence | Every commit / push via CI; continuous |
| Who is involved | Separate test team; business analysts for acceptance | Whole team; devs, QA, product owner; QA embedded |

Which model our plan follows: Agile/DevOps. The one-week cadence, the CI pipeline that runs on every push, and the pytest suite all match continuous integration. We still design the state diagram, EP/BVA tables and decision table up front. a V-model habit, but execute them continuously. This is a deliberate hybrid leaning agile.

### A4 evidence summary

| A4 requirement | Evidence | Location |
|---|---|---|
| Project structure(src/ ,tests/) | Repository tree visible on GitHub | github.com/micheal-61/tickethub-qa |
| Dependency file | requirements.txt(pytest 8.2.0, pytest-html 4.1.1) | Root of repo |
| README with run command | README.md | Root of repo |
| Build/task definition | Makefile with install + test targets | Root of repo |
| CI pipeline | .github/workflows/ci.yml | Installs dependencies, runs tests, uploads JUnit XML |
| Evidence of CI runs | 4 workflow runs, all green | Actions tab + ci/ci-run-log.txt + ci/ci-run.png |
| Published test report | JUnit XML artifact + committed XML | ci/test-results.xml |

## Part B: The Testing Process and Test Levels

## B1. Requirements Review

| REQUIREMENT | ISSUE | FIX |
|---|---|---|
| F1 | "1 to 10 inclusive" vs interface showing 10 returns; boundary 10 ambiguous | State: "10 is valid; only >10 raises ValueError." |

| | | |
|---|---|---|
| F4 | Additive formula contradicts "never combine" | Rewrite: additional = promo if given, else bulk if q≥5, else 0 |
| F5 | Half-up stated but no worked example | Add example: 47.976 → 47.98 |
| F6 | No rule for whitespace / mixed case beyond "case-sensitive" | State: exact string equality, no trim |
| F7 | Table shows PAID→cancel→REFUND_PENDING, but also implies CANCELLED is reachable — contradiction | Clarify: CANCELLED only from CREATED |
| N1 | "fast and user-friendly" untestable | Rewrite: 200 ms p95 at 500 users (see below) |
| N2 | 200 ms / 500 users; no hardware, tool, or window given | Specify: Locust, 60s run, 2 vCPU / 4 GB runner |

**Rewritten REQ-N1:** calculate_total shall return in < 200 ms for 95% of calls over a 60-second run at 500 concurrent users on a 2 vCPU / 4 GB runner, measured with Locust.

### B2. Test Levels and Types

| ID | FEATURE | LEVEL / TYPE | JUSTIFICATION |
|---|---|---|---|
| F1 | Price calculation | Unit / functional | Pure function; isolated inputs and outputs |
| F2 | Order lifecycle | Unit + integration (functional) | State machine is unit-testable; must also be checked end-to-end. |
| F3 | Checkout page calling pricing module | Integration + system / functional | Verifies UI→module boundary |
| F4 | order.pay() calling payment gateway | Integration / functional | Third-party contract; needs stubs and error-path tests |
| F5 | 500 concurrent buyers responsiveness | System / non-functional (performance) | Load is a system property |
| F6 | Organiser confirms discount policy | Acceptance / functional | Only the stakeholder can sign off. |

The checkout page sending e.g., “3” (string) instead of 3 (int) should be caught at the integration level. A unit test of calculate_total in isolation passes an int directly and never exercises the web layer's type handling. The integration test proves the system wires the layers correctly.

### B3. Test Plan

### Test Plan — TicketHub Order Module v0.9

### 1. Scope

`calculate_discount`, `calculate_total`, `Order` state machine, validation, rounding, discount cap, promo handling, boundary and equivalence-class coverage.
**Out of scope:** payment gateway internals, web-UI rendering, database persistence, REQ-N2 performance (planned but not executed this week).

**2. Items under test**

src/tickethub.py (unchanged, checksum recorded).

Python 3.10+, pytest 8.2.0, pytest-html 4.1.1, Decimal for money.
CI runner: Ubuntu latest via GitHub Actions, Python 3.12.

**3. Test approach**

**Unit / functional:** EP, BVA, decision table, state transition → tests/test_tickethub.py.
**Integration / functional:** Order + pricing combined (planned).
**System / non-functional:** REQ-N2 performance using Locust (planned, not executed this week).
**Acceptance:** manual sign-off by the event organiser on the collapsed decision table.

**4. Entry criteria**

tickethub.py frozen and checksummed.
Requirements reviewed (B1) and ambiguities logged.
Test cases designed, reviewed and stored in test_design/test_cases.csv.

**5. Exit criteria**

100% of designed test cases executed.
Every REQ-F1..F7 covered by at least one passing test.
No open severity-1 or severity-2 defects.
Defect density and requirement coverage reported in the summary (Part E).
Traceability matrix complete.

**6. Roles and responsibilities**

**QA engineer (you):** design, automate, execute, report.
**Lecturer / product owner:** review plan, arbitrate requirement ambiguities, sign off acceptance.

**7. Test environment**

Local: Windows 11, Python 3.14.8 inside a .venv, pytest 8.2.0.
CI: GitHub Actions, Ubuntu-latest, Python 3.12, JUnit XML artifact published.
Both environments install the same pinned requirements.txt.

**8. Schedule** (based on the one-week plan)

Day 1–2: setup, requirements review, plan.
Day 3: EP + BVA design.
Day 4: decision table + state transition.
Day 5: automate, run locally and in CI.
Day 6: defect reports + summary report.
Day 7: traceability, packaging, submission.

9. Risks and mitigation

| Risk | Mitigation |
|---|---|
| Requirements ambiguous (REQ-F4, REQ-F7) | B1 review; log ambiguity as a defect |
| White-box bias | Design all cases from Appendix A before reading tickethub.py |
| Rounding disputes | Decimal + hand-worked examples |

| | |
|---|---|
| CI flakiness | Pinned versions; deterministic tests; xfail (strict = True) |

## 10. Deliverables

- tests/test_tickethub.py(pytest suite)
- test_design/test_cases.csv (test-case sheet)
- defects/DEF-01.md..DEF-08.md
- ci/test-results.xml, ci/report.html
- report/SQT_Test1.pdf
- README.md

## B4. Traceability Matrix

| Requirement | Testcase IDs | Notes |
|---|---|---|
| REQ-F1 | TC-01 … TC-08 | EP + BVA on quantity; 10 accepted, 11 rejected, non-integers TypeError |
| REQ-F2 | TC-09 … TC-12 | EP on customer type |
| REQ-F3 | TC-13, TC-14 | Unit price constant 19.99 |
| REQ-F4 | TC-15 … TC-27 | Decision table (9 collapsed rules) + cap + promo/bulk exclusivity |
| REQ-F5 | TC-28 … TC-31 | Rounding half-up; exact totals |
| REQ-F6 | TC-32 … TC-35 | Promo case-sensitivity; invalid codes |
| REQ-F7 | TC-36 … TC-60 | State transition: valid + invalid + length≥2 paths |
| REQ-N1 | - | **Untestable as written** (subjective). Rewritten → REQ-N1′ |
| REQ-N1′ | TC-61 (planned) | Performance: 200 ms p95 at 500 users (Locust) |
| REQ-N2 | TC-61 (planned) | Same test also covers REQ-N2 |

No functional requirement (REQ-F1..F7) is uncovered. REQ-N1 is untestable as written and has been rewritten to REQ-N1′, mapped to TC-61.

## Part C: Black-Box Test Design

## C1. Equivalence Partitioning

### Quantity (REQ-F1):

| Class | Range | Representative | Expected |
|---|---|---|---|
| valid | 1 - 10 | 4, 7 | accepted |
| invalid low | < 1 | 0 | ValueError |
| invalid high | > 10 | 11 | ValueError |
| Invalid type | Non-integer | 2.5, "3", None, True | TypeError |

### Customer type (REQ-F2):

| CLASS | REPRESENTATIVES | EXPECTED |
|---|---|---|
| STUDENT | "STUDENT" | 20% |
| STAFF | "STAFF" | 10% |
| PUBLIC | "PUBLIC" | 0% |

| | | |
|---|---|---|
| invalid | "student", "ALUMNI", "", None | ValueError |

**Promo code (REQ-F6):**

| CLASS | REPRESENTATIVES | EXPECTED |
|---|---|---|
| None | None | No promo |
| SAVE10 | “SAVE10” | 10% |
| Invalid | “BADCODE”, “ ”, “save10”, “SAVE20”. | ValueError |

**Rationale:** 4 and 7 isolate bulk effect (below / above threshold). One representative per invalid class, tested one at a time. Lowercase "save10" tests case-sensitivity specifically.

### C2. Boundary Value Analysis

**Quantity range 1..10 (three-value BVA):**

| BOUNDARY | VALUE | PARTITION | EXPECTED |
|---|---|---|---|
| Lower−1 | 0 | Invalid | ValueError |
| Lower | 1 | Valid | Accepted |
| Lower +1 | 2 | Valid | Accepted |
| Upper -1 | 9 | Valid | Accepted |
| Upper | 10 | Valid | Accepted |
| Upper +1 | 11 | Invalid | ValueError |

**Bulk threshold 5 (three-value BVA):**

| BOUNDARY | VALUE | PARTITION | EXPECTED RATE (PUBLIC) |
|---|---|---|---|
| Below | 4 | No bulk | 0.00 |
| At | 5 | Bulk | 0.05 |
| Above | 6 | Bulk | 0.05 |

**Why no BVA for customer type / promo code:** both are nominal (unordered) categories, not numeric. BVA applies only to ordered or continuous inputs. Categories use equivalence partitioning (C1).

### C3. Decision Table and Cause-Effect Graph

**Conditions:**

**C1: customer type = STUDENT / STAFF / PUBLIC**
**C2: quantity ≥ 5 (bulk)**
**C3: valid promo supplied**

## Full decision table

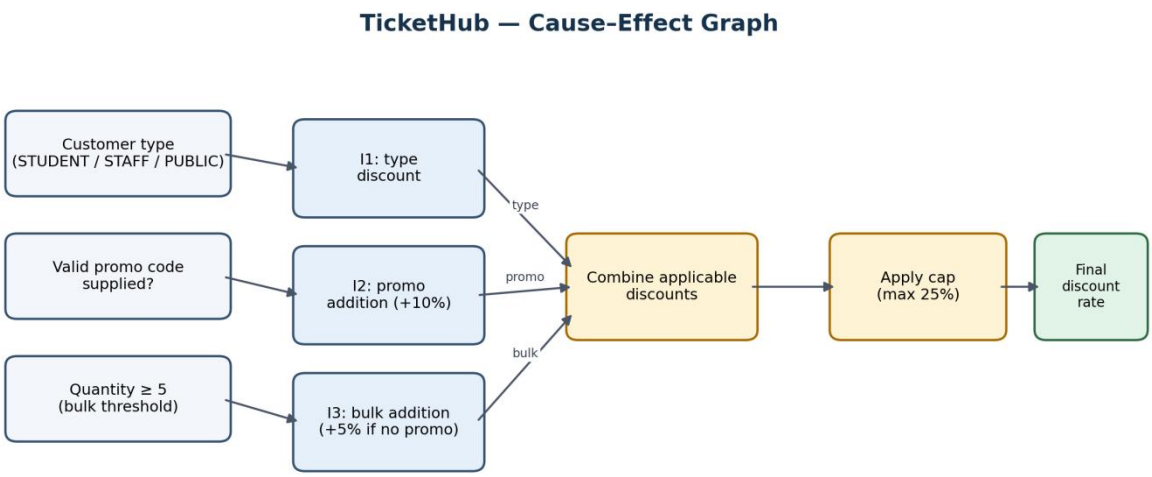
| RULE | C1 | C2 | C3 | ADDITIONAL | RATE |
|---|---|---|---|---|---|
| R1 | STUDENT | N | N | 0 | 0.00 |
| R2 | STUDENT | Y | N | 0.05 | 0.25 |
| R3 | STUDENT | N | Y | 0.10 | 0.25(cap) |
| R4 | STUDENT | Y | Y | 0.10 | 0.25(cap) |
| R5 | STAFF | N | N | 0 | 0.10 |
| R6 | STAFF | Y | N | 0.05 | 0.15 |
| R7 | STAFF | N | Y | 0.10 | 0.20 |
| R8 | STAFF | Y | Y | 0.10 | 0.20 |
| R9 | PUBLIC | N | N | 0 | 0.00 |
| R10 | PUBLIC | Y | N | 0.05 | 0.05 |
| R11 | PUBLIC | N | Y | 0.10 | 0.10 |
| R12 | PUBLIC | Y | Y | 0.10 | 0.10 |

## Collapsed table

| RULE | TYPE | q≥5 | PROMO | RATE |
|---|---|---|---|---|
| 1 | STUDENT | N | N | 0.20 |
| 2 | STUDENT | Y | N | 0.25 |
| 3 | STUDENT | - | Y | 0.25 |
| 4 | STAFF | N | N | 0.10 |
| 5 | STAFF | Y | N | 0.15 |
| 6 | STAFF | - | Y | 0.20 |
| 7 | PUBLIC | N | N | 0.00 |
| 8 | PUBLIC | Y | N | 0.05 |
| 9 | PUBLIC | - | Y | 0.10 |

No information lost: when a promo is present, C2 is irrelevant (promo supersedes bulk; cap covers STUDENT+promo).

## Cause-effect graph




When a valid promo is supplied, the bulk addition does not apply; the final rate is capped at 25%.

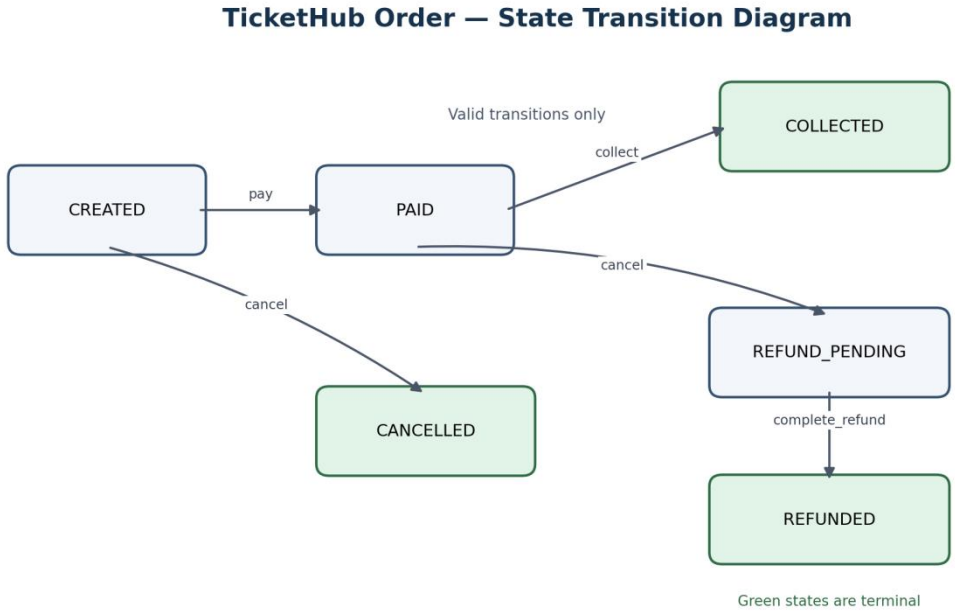
### Test cases — one per collapsed rule (non-boundary quantities)

| RULE | q | TYPE | PROMO | RATE | TOTAL |
|---|---|---|---|---|---|
| 1 | 4 | STUDENT | None | 0.20 | 4×19.99×0.80 = 63.968 → **63.97** |
| 2 | 7 | STUDENT | None | 0.25 | 7×19.99×0.75 = 104.9475 → **104.95** |
| 3 | 4 | STUDENT | SAVE10 | 0.25 | 4×19.99×0.75 = 59.97 → **59.97** |
| 4 | 4 | STAFF | None | 0.10 | 4×19.99×0.90 = 71.964 → **71.96** |
| 5 | 7 | STAFF | None | 0.15 | 7×19.99×0.85 = 118.9405 → **118.94** |
| 6 | 7 | STAFF | SAVE10 | 0.20 | 7×19.99×0.80 = 111.944 → **111.94** |
| 7 | 4 | PUBLIC | None | 0.00 | 4×19.99 = 79.96 → **79.96** |
| 8 | 7 | PUBLIC | None | 0.05 | 7×19.99×0.95 = 132.9335 → **132.93** |
| 9 | 4 | PUBLIC | SAVE10 | 0.10 | 4×19.99×0.90 = 71.964 → **71.96** |

Rule 3 explicitly covers the cap (20% + 10% = 30%, capped at 25%).

## C4. State Transition Testing

### State transition diagram



### Full state × event table

| STATE / EVENT | pay | cancel | collect | complete_refund |
|---|---|---|---|---|
| CREATED | PAID | CANCELLED | INVALID | INVALID |
| PAID | INVALID | REFUND_PENDING | COLLECTED | INVALID |
| COLLECTED | INVALID | INVALID | INVALID | INVALID |
| REFUND_PENDING | INVALID | INVALID | INVALID | REFUNDED |
| REFUNDED | INVALID | INVALID | INVALID | INVALID |
| CANCELLED | INVALID | INVALID | INVALID | INVALID |

**Coverage criterion:** all states + all valid transitions + all invalid (state, event) pairs = full transition coverage (0-switch).

### Test cases: valid transitions

| TC | START | EVENT | END |
|---|---|---|---|
| 36 | CREATED | pay | PAID |
| 37 | CREATED | cancel | CANCELLED |
| 38 | PAID | cancel | REFUND_PENDING |
| 39 | PAID | collect | COLLECTED |
| 40 | REFUND_PENDING | complete_refund | REFUNDED |

### Test cases: invalid combinations

TC-41 … TC-59: every cell marked invalid above. Each asserts InvalidTransitionError and that state is unchanged.

### Length ≥2 sequences

| TC | Sequence | Final |
|---|---|---|
| 60a | pay → collect | COLLECTED |
| 60b | pay → cancel → complete_refund | REFUNDED |

# Part D: Automation, Execution and Defect Reports

## D1. Automated suite

tests/test_tickethub.py — 68 collected tests, traceable to TC IDs via test names and to REQ IDs via inline comments. Parameterised where the same check runs over multiple data rows. Monetary values are compared with Decimal (exact), never floats.

## D2. Execution in CI

Local run and GitHub Actions CI run:

- ci/ci-run-log.txt — green run log
- ci/ci-run.png — green run screenshot
- ci/ci-run-failing.txt — failing run log (19 tests exposed defects)
- ci/ci-run-failing.png — failing run screenshot
- ci/test-results.xml — JUnit XML artifact

Failing tests tied to defects are marked @pytest.mark.xfail(strict=True, reason="DEF-0n"), so the pipeline stays green while defects are open and turns red again when a defect is fixed (strict mode).

## D3. Defect reports

Eight distinct root-cause defects are logged in defects/DEF-01.md through defects/DEF-08.md. Each report records reproduction steps, expected versus actual behaviour, severity, priority, and suspected root cause. See Part E2 for the phase-introduced / phase-found analysis.

## Part E: Test Closure

### E1. Execution summary and coverage

Executed the pytest suite against src/tickethub.py locally and in GitHub Actions CI. Results: **44 passed, 19 expected failures (xfail), 0 unexplained failures** across 63 executed tests (plus 5 skips for valid state-pairs). All eight open defects are tied to specific xfail markers, so the pipeline turns red automatically once a defect is fixed.

| Metric | value |
|---|---|
| Planned test cases | 61 (TC-01 … TC-61) |
| Executed | 63 parametrized rows |
| Passed | 44 |
| Expected failures (xfail) | 19 |
| Unexplained failures | 0 |
| Skipped | 5 |
| Requirement coverage (REQ-F1..F7) | 7/7 (100%) |
| Requirement coverage (REQ-N1) | Untestable as written; rewritten to REQ-N1′ |
| Requirement coverage (REQ-N2) | Planned, not executed this week |
| Defects logged | 8 distinct root causes (DEF-01 … DEF-08) |

### E2. Defect escape and cost analysis

| Defect | Phase introduced | Phase found | Cheapest phase to catch | Technique that catches it |
|---|---|---|---|---|
| DEF-01 (q=10 rejected) | Implementation | Test execution | Unit test | Boundary value analysis |
| DEF-02 (bulk at 6) | Requirements/implementation | Test execution | Requirements review | Decision table (boundary 5) |
| DEF-03 (cap 30%) | Requirements | Test execution | Requirements review | Decision table |
| DEF-04 (promo case) | Implementation | Test execution | Unit test | Equivalence partitioning |
| DEF-05 (promo+bulk stack) | Requirements | Test execution | Requirements review | Decision table |
| DEF-06 (ROUND_DOWN) | Implementation | Test execution | Unit test | Rounding test with hand-worked example |
| DEF-07 (PAID+cancel) | Requirements/design | Test execution | Design review | State transition testing |
| DEF-08 (COLLECTED+cancel allowed) | Implementation | Test execution | Design review | State transition testing |

The most severe defect is **DEF-07** (PAID + cancel → CANCELLED instead of REFUND_PENDING). It is critical because money has been taken from the buyer, the order is marked CANCELLED (a terminal state), and there is no longer any path to refund the payment. If this reached production, the cost would be significant: every paid order that is cancelled would need manual reconciliation, a refund issued outside the system, and the accounting ledger corrected. In a real ticketing platform, this would also trigger customer complaints, chargebacks, and potentially regulatory issues. Fixing DEF-07 in requirements review — where the state model was being defined — would have cost minutes. Finding it in production costs hours of manual refund work per order plus reputational damage. The lesson: state machines should be reviewed at design time, before any code is written.

### E3. Release recommendation

**Recommendation: NO-GO for production; CONDITIONAL GO for a limited internal pilot.**

Against the exit criteria defined in B3, two conditions are not yet satisfied:

- "No open severity-1 or severity-2 defects" — DEF-03, DEF-05, DEF-07 and DEF-08 are severity-2 or higher.
- All REQ-F1..F7 covered by ≥1 passing test — true, but 19 rows remain as expected failures tied to open defects.

The module is **not fit for production release** in its current state. It **is** fit for a **limited internal pilot** with discounts hard-coded to the correct values as a workaround. Once DEF-03, DEF-05, DEF-07 and DEF-08 are fixed and the xfail markers removed (making the pipeline red-then-green again), the module can be promoted to release.

## E4. Reflection

### Is testing the same as quality assurance?

No. Testing is one QC activity that finds defects in a product; QA is the broader set of process activities that prevent defects from being introduced in the first place. This week demonstrated the difference clearly. When I reviewed Appendix A in B1, I found ambiguities (REQ-F4's additive-vs-exclusive wording) and a contradiction (REQ-F7's CANCELLED reachability) that no amount of test execution could resolve — those are QA findings, and they are cheap. When I later executed the pytest suite and found DEF-03 (30% cap) and DEF-05 (stacking), those were QC findings — necessary, but reactive. The eight defects fell into two rough buckets: those a review could have caught (DEF-02, DEF-03, DEF-05, DEF-07, DEF-08) and those only execution revealed (DEF-01, DEF-04, DEF-06). QA can prevent most of the first bucket; testing will always be needed for the second.

**A quality concern my approach could not have detected:** I designed tests only from the requirements. If the requirements themselves are wrong — for example, if the organiser actually wanted the promo and bulk discounts to stack, and REQ-F4 mis-stated the rule — my suite would happily pass a system that does the wrong thing. That is a validation failure, not a verification one, and no amount of black-box testing against the spec can find it. Only acceptance testing with the stakeholder can.